BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch:1 Practical No: 3

Roll no: 11 PRN No: 12413071 Name of Student: Patil Shreyas Prashant

Subject Name : Deep Learning

Problem Statement :

Implement forward propagation and backpropagation using TensorFlow/Keras. Analyze the effect of different learning rates and the number of epochs on model performance.

Algorithm :

1. Start the program.
2. Import the required libraries such as TensorFlow, Keras, NumPy, Pandas, and Matplotlib.
3. Load the MNIST dataset containing handwritten digit images.
4. Preprocess the dataset by:
   - Normalizing pixel values to the range 0–1.
   - Reshaping each 28 × 28 image into a 784-dimensional input vector.
5. Define the neural network architecture using Keras Sequential model:
   - Input layer with 784 features.
   - Hidden Layer 1 with 256 neurons and ReLU activation.
   - Hidden Layer 2 with 128 neurons and ReLU activation.
   - Output layer with 10 neurons and Softmax activation.
6. Compile the model using:
   - Adam Optimizer (implements backpropagation for weight updates).
   - Sparse Categorical Crossentropy as the loss function.
   - Accuracy as the evaluation metric.
7. Train the model using forward propagation and backpropagation:
   - Perform forward propagation to compute predictions.
   - Calculate the loss between predicted and actual labels.
   - Perform backpropagation to compute gradients.
   - Update weights using the Adam optimizer.

- Repeat the process for all batches and epochs.

8. Analyze different learning rates (e.g., 0.1, 0.01, 0.001) by:
   - Training a new model for each learning rate.
   - Recording the test loss and accuracy.
   - Comparing the results.

9. Analyze different numbers of epochs (e.g., 5, 10, 20) by:
   - Training the model for each epoch value.
   - Evaluating model performance.
   - Comparing the obtained accuracies.

10. Visualize the results by plotting:
    - Learning Rate vs Accuracy.
    - Epochs vs Accuracy.
    - Training Accuracy vs Validation Accuracy.

11. Evaluate the final model on the test dataset and display the final loss and accuracy.

12. Analyze the observations to determine the optimal learning rate and number of epochs.

13. Stop the program.

Code :

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

print("TensorFlow Version:", tf.__version__)

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

x_train = x_train / 255.0
x_test = x_test / 255.0

x_train = x_train.reshape(-1, 784)
x_test = x_test.reshape(-1, 784)

print("Training Shape:", x_train.shape)
print("Testing Shape :", x_test.shape)

def build_model(learning_rate):
```

```
    model = Sequential([
        Dense(256, activation='relu', input_shape=(784,)),
        Dense(128, activation='relu'),
        Dense(10, activation='softmax')
    ])

    optimizer = Adam(learning_rate=learning_rate)

    model.compile(
        optimizer=optimizer,
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    return model


learning_rates = [0.1, 0.01, 0.001]

lr_results = []

for lr in learning_rates:

    print(f"\nTraining with Learning Rate = {lr}")

    model = build_model(lr)

    history = model.fit(
        x_train,
        y_train,
        epochs=5,
        batch_size=128,
        verbose=0
    )

    loss, accuracy = model.evaluate(
        x_test,
        y_test,
        verbose=0
    )

    lr_results.append([lr, loss, accuracy])

lr_df = pd.DataFrame(
    lr_results,
    columns=["Learning Rate", "Loss", "Accuracy"]
)

print(lr_df)
```

```
plt.figure(figsize=(7,5))

plt.plot(
    lr_df["Learning Rate"],
    lr_df["Accuracy"],
    marker='o',
    linewidth=2
)

plt.xscale("log")

plt.xlabel("Learning Rate")
plt.ylabel("Accuracy")
plt.title("Learning Rate vs Model Accuracy")
plt.grid(True)

plt.show()


epochs_list = [5, 10, 20]

epoch_results = []

for ep in epochs_list:

    print(f"\nTraining for {ep} Epochs")

    model = build_model(0.001)

    history = model.fit(
        x_train,
        y_train,
        validation_split=0.2,
        epochs=ep,
        batch_size=128,
        verbose=0
    )

    loss, accuracy = model.evaluate(
        x_test,
        y_test,
        verbose=0
    )

    epoch_results.append([ep, loss, accuracy])

epoch_df = pd.DataFrame(
    epoch_results,
    columns=["Epochs", "Loss", "Accuracy"]
)
```

```
print(epoch_df)

plt.figure(figsize=(7,5))

plt.plot(
    epoch_df["Epochs"],
    epoch_df["Accuracy"],
    marker='s',
    linewidth=2
)

plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.title("Epochs vs Model Accuracy")
plt.grid(True)

plt.show()

best_model = build_model(0.001)

history = best_model.fit(
    x_train,
    y_train,
    validation_split=0.2,
    epochs=10,
    batch_size=128,
    verbose=1
)

test_loss, test_accuracy = best_model.evaluate(
    x_test,
    y_test
)

print("\nFinal Test Accuracy:", round(test_accuracy * 100, 2), "%")

plt.figure(figsize=(8,5))

plt.plot(history.history['accuracy'], label='Training Accuracy')

plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.xlabel("Epoch")

plt.ylabel("Accuracy")

plt.title("Training vs Validation Accuracy")

plt.legend()

plt.grid(True)
```

plt.show()

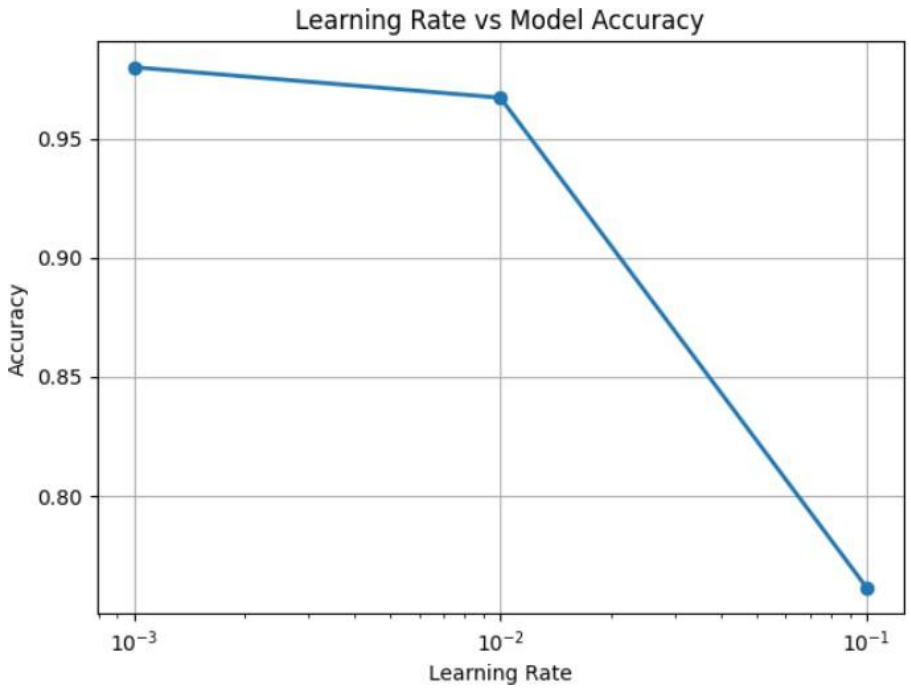
Output :

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training Shape: (60000, 784)
Testing Shape : (10000, 784)
```

```
Training with Learning Rate = 0.1
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Training with Learning Rate = 0.01

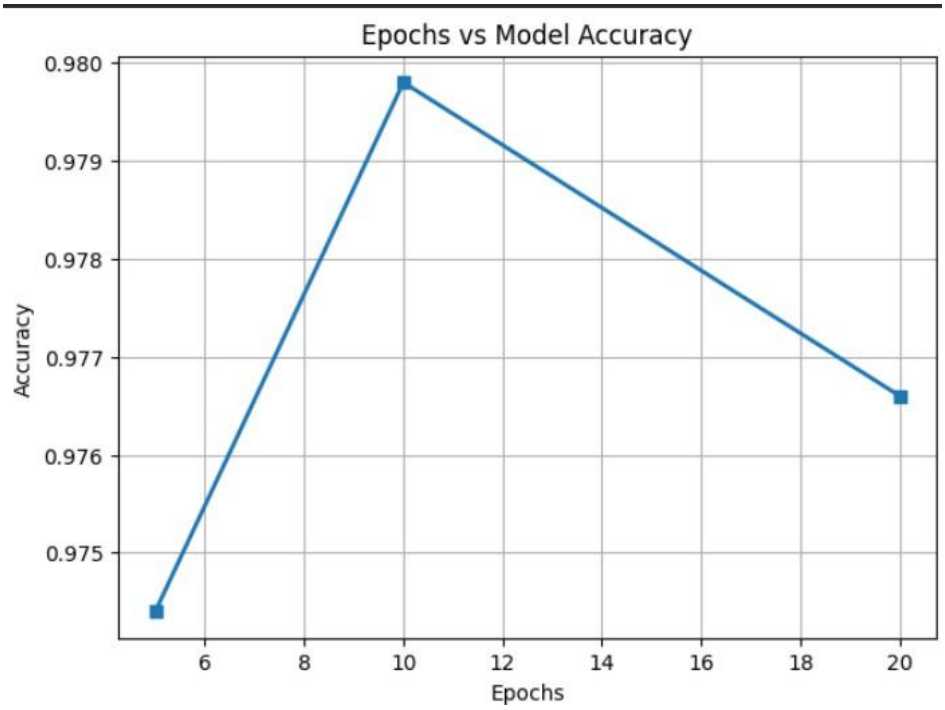
Training with Learning Rate = 0.001
   Learning Rate      Loss  Accuracy
0          0.100  0.803789    0.7614
1          0.010  0.132319    0.9671
2          0.001  0.067331    0.9799
```



```
Training for 5 Epochs
/usr/local/lib/python3.12/dist-packages/keras/src/layers/cor
  super().__init__(activity_regularizer=activity_regularizer

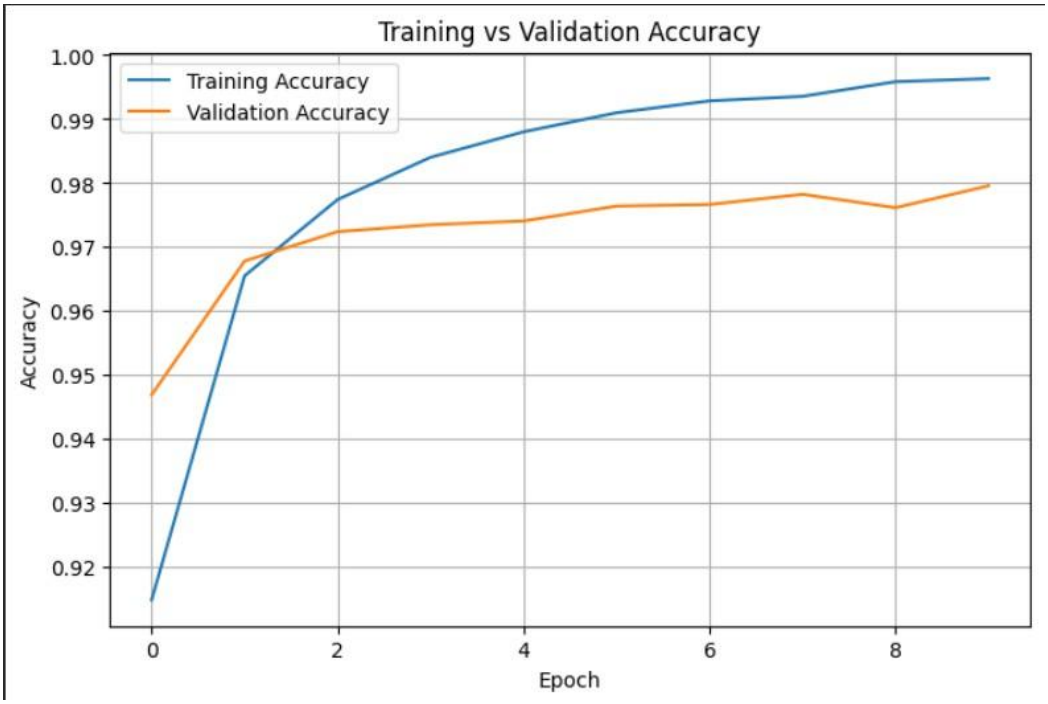
Training for 10 Epochs

Training for 20 Epochs
   Epochs      Loss  Accuracy
0       5  0.082338    0.9744
1      10  0.084201    0.9798
2      20  0.113344    0.9766
```




```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_di
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 9ms/step - accuracy: 0.9149 - loss: 0.3069 - val_accuracy: 0.9469 - val_loss: 0.1751
Epoch 2/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - accuracy: 0.9655 - loss: 0.1149 - val_accuracy: 0.9678 - val_loss: 0.1074
Epoch 3/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 7ms/step - accuracy: 0.9774 - loss: 0.0746 - val_accuracy: 0.9724 - val_loss: 0.0923
Epoch 4/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - accuracy: 0.9840 - loss: 0.0526 - val_accuracy: 0.9735 - val_loss: 0.0853
Epoch 5/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - accuracy: 0.9880 - loss: 0.0388 - val_accuracy: 0.9741 - val_loss: 0.0881
Epoch 6/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.9910 - loss: 0.0289 - val_accuracy: 0.9764 - val_loss: 0.0887
Epoch 7/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - accuracy: 0.9929 - loss: 0.0229 - val_accuracy: 0.9767 - val_loss: 0.0842
Epoch 8/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - accuracy: 0.9936 - loss: 0.0199 - val_accuracy: 0.9783 - val_loss: 0.0832
Epoch 9/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - accuracy: 0.9959 - loss: 0.0133 - val_accuracy: 0.9762 - val_loss: 0.0871
Epoch 10/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - accuracy: 0.9963 - loss: 0.0114 - val_accuracy: 0.9796 - val_loss: 0.0890
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9803 - loss: 0.0799

Final Test Accuracy: 98.03 %
```


Training vs Validation Accuracy
Training Accuracy
Validation Accuracy
Accuracy
Epoch
1.00
0.99
0.98
0.97
0.96
0.95
0.94
0.93
0.92
0
2
4
6
8